

22. Bezkontextové jazyky a jejich modely (zásobníkové automaty, bezkontextové gramatiky)

22. Bezkontextové jazyky a jejich modely

SZZ okruh 22 (FIT BUT). Silnější třída než [[okruhy/21-regularni-jazyky | regulární jazyky]]
— gramatika + zásobníkový automat.

Shrnutí

Bezkontextová gramatika

- $G = (N, T, P, S)$ — neterminály, terminály ($N \cap T = \emptyset$), pravidla $A \rightarrow x$ ($A \in N, x \in (N \cup T)^*$), počáteční symbol.
- Typický BKJ: $L = \{a^n b^n \mid n \geq 0\}$ (neregulární).
- **Derivace** (nejlevější/nejpravější), **derivační strom**.

Zásobníkový automat

- $M = (Q, \Sigma, \Gamma, R, s, S, F)$ — konečný automat + **zásobník**; pravidlo $A_p \rightarrow w_q$.
- Přijetí: **koncovým stavem, vyprázdněním zásobníku**, nebo obojím — všechny **ekvivalentní**.
- **DZA** (deterministický) je **slabší** než nedeterministický ZA. **RZA** (čte celý řetězec z vrcholu) je ZA ekvivalentní.

Srovnání jazyků

- $RJ \subset BKJ$. Příklady: a^* (oba), $a^n b^n$ (jen BKJ), $a^n b^n c^n$ (ani jeden).
- ZA/RZA **simulují konstrukci derivačního stromu** pro [[okruhy/23-struktura-prekladace | syntaktickou analýzu]] (shora dolů = expanze, zdola nahoru = shift/redukce).

Související syntéza

- [[synthesis/konecne-automaty-napric-obory | Konečné automaty napříč obory]] — syntéza
- [[synthesis/zasobnik-napric-obory | Zásobník napříč obory]] — syntéza

Časté otázky u zkoušky

Na co se u tohoto okruhu typicky ptají. Plné odpovědi níže.

Bezkontextová gramatika $\square$ [[#Bezkontextová gramatika]] - *Definice, tvar pravidla?* $\square$ Čtveřice (N, T, P, S) ; pravidlo $A \rightarrow x$, kde A je neterminál, x řetězec terminálů/neterminálů. - *Co generuje, příklad?* $\square$ Množinu řetězců derivovatelných z S ; klasicky $a^n b^n$. - *Derivační strom / krok?* $\square$ Postupné přepisování neterminálů (nejlevější/nejpravější derivace).

Zásobníkový automat $\square$ $[[\#Zásobníkový\ automat]]$ - *Definice (sedmice)?* $\square (Q, \Sigma, \Gamma, R, s, S, F)$ — konečný automat se zásobníkem. - *Jak přijímá jazyk?* $\square$ koncovým stavem / vyprázdněním zásobníku / obojím (ekvivalentní).

Srovnání $\square$ $[[\#Srovnání\ jazyků]]$ - *RJ vs. BKJ?* $\square$ Každý RJ je BKJ, ne naopak ($a^n b^n$ je BKJ, ne RJ); $a^n b^n c^n$ není ani BKJ. - *Proč „bezkontextový“?* $\square$ Pravidlo přepisuje neterminál bez ohledu na okolní kontext.

Plné znění (ke studiu)

Bezkontextový jazyk (BKJ)

Jedná se o formální jazyk, pro který platí:

- **Generuje** jej **bezkontextová gramatika**,
- **Přijímá** (akceptuje) jej **zásobníkový automat**.

Bezkontextová gramatika

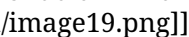
Slouží pro **generování BKJ**. Je to **čtveřice** $G = (N, T, P, S)$, kde:

- **N** - abeceda **neterminálů**,
- **T** - abeceda **terminálů**, přičemž $N \cap T = \emptyset$,
- **P** - konečná **množina pravidel** tvaru $A \rightarrow x$,
kde $A \in N$, $x \in (N \cup T)^*$,
- **S** - počáteční neterminál, $S \in N$.

Nejznámějším bezkontextovým jazykem je jazyk

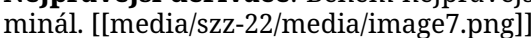
$L(G) = \{(a^n)^0 b^n : n \geq 0\}$ 

Derivační krok BKG

Jedná se o změnění řetězce použitím pravidla na neterminál. Pokud $uAv \in uxv$ v G , můžeme říct, že G provádí derivační krok z uAv do uxv . 





- **Nejlevější derivace:** Během nejlevějšího derivačního kroku je přepsán nejlevější neterminál.
- **Nejpravější derivace:** Během nejpravějšího derivačního kroku je přepsán nejpravější neterminál. 





Zásobníkový automat

Konečný automat rozšířený o **zásobník**. Pro každou **BKG existuje** zásobníkový **automat**, který **ji přijímá**. Zásobníkový automat (ZA) je **sedmice**:

$M = (Q, \Sigma, \Gamma, R, s, S, F)$, kde:

- **Q** je konečná **množina stavů**,
- Σ je **vstupní abeceda**,

- Γ je zásobníková abeceda,
- R je konečná množina pravidel tvaru $A \rightarrow wq$, kde $A \in \Gamma$; $p, q \in Q$;
 $a \in \Sigma \setminus \{\epsilon\}$, $w \in \Gamma^*$, (R je konečná relace z $\Gamma \times Q \times (\Sigma \setminus \{\epsilon\})$ do $\Gamma^* \times Q$)
- $s \in Q$ je počáteční stav,
- $S \subseteq \Gamma$ je počáteční symbol na zásobníku,
- $F \subseteq Q$ je množina koncových stavů.

Interpretace pravidel: $A \rightarrow wq \in R$ znamená, že pokud je aktuální stav p , aktuální symbol na vstupní pásce a a symbol na vrcholu zásobníku A , potom zásobníkový automat může přečíst a a na zásobníku nahradit A za w a přejít ze stavu p do q . 



Typy přijímaných jazyků ZA

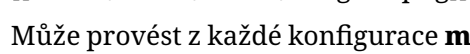
- **Přechodem do koncového stavu:** $ZA M$ přijímá jazyk L , pokud se čtením všech řetězců jazyka L dostane do koncového stavu.
- **Vyprázdněním zásobníků:** $ZA M$ přijímá jazyk L , pokud se čtením všech řetězců jazyka L vyprázdní jeho zásobník.
- **Přechodem do koncového stavu a vyprázdněním zásobníku:** přijímá jazyk L , pokud se čtením všech řetězců jazyka L dostane do koncového stavu **a současně** se vyprázdní jeho zásobník.

Tyto tři typy ZA jsou si **ekvivalentní** a **existují** algoritmy pro **převody** mezi nimi.

Konfigurace ZA

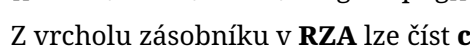
Konfigurace $ZA M$ je řetězec $\chi \in \Gamma^* Q \Sigma^*$. Jedná se o **aktuální stav zásobníku**, **aktuálního stavu** a **vstupní pásky**, jejíž část **nebyla** ještě **přečtena**.

Deterministický zásobníkový automat (DZA)



Může provést z každé konfigurace **maximálně jeden přechod**. M je deterministický ZA , pokud pro každé pravidlo tvaru $A \rightarrow wq \in R$ platí, že množina $R \setminus \{A \rightarrow wq\}$ (množinový rozdíl) neobsahuje **žádné pravidlo** s levou stranou A nebo A . Jinak řečeno, **levá strana** pravidel je v množině R vždy **pouze jednou** – je **unikátní**. DZA je **podmnožinou** ZA , DZA nemusí přijímat některé jazyky, která ZA přijímá $\Rightarrow ZA$ je **silnější než** DZA .

Rozšířený zásobníkový automat (RZA)



Z vrcholu zásobníku v RZA lze číst **celý řetězec** (v ZA to byl pouze jeden symbol). Pravidla jsou tak definována takto: R je **konečná** množina pravidel tvaru: $vpa \rightarrow wq$, kde $v, w \in \Gamma^*$; $p, q \in Q$; $a \in \Sigma \setminus \{\epsilon\}$. RZA je **ekvivalentní** s ZA (jsou stejně silné).

Mohou existovat RZA , které **přijímají jazyky přechodem do koncového stavu, vyprázdněním zásobníku** nebo vyprázdněním zásobníku a současně přechodem do koncového stavu. Opět jsou tyto typy RZA ekvivalentní. Následuje příklad RZA . 



RZA a ZA jako modely pro synt. analýzu

RZA nebo **ZA** mohou **SIMULOVAT konstrukci derivačního stromu** pro **BKG**. Používají se k tomu dva přístupy: 

Převod z BKG na RZA (pro SA zdola nahoru)

1. obrázek je pro SA zdola nahoru, 2. pro SA shora dolů.

Např. (pořadí obrázků zůstává): 









Modely pro syntaktickou analýzu shora dolů

ZA $M = (Q, \Sigma, \Gamma, R, s, S, F)$ (F = prázdná množina) a **BKG** $G = (N, T, P, S)$

1. **M** obsahuje **porovnávací pravidla**, která **porovnají** symbol z vrcholu **zásobníku** a aktuální symbol ze **vstupní pásky**. (tvorba: pro každé $a \in \Sigma$: přidej a do R)
2. **M** obsahuje **expanzivní pravidla**, která **simulují gramatická pravidla**. (tvorba: pro každé $A \in T$ přidej A do R ; = **reversal**($t_1 \dots t_n$))

Jinak řečeno:

Z počátečního neterminálu **S**, který přepisujeme na jiné neterminály a terminály, se snažíme dostat k řetězci, který načítáme. Potřebujeme k tomu dva druhy pravidel:

1. Pravidla, které přepíší neterminál z vrcholu zásobníků na jiný neterminál/neterminály nebo přímo načítané znaky řetězce, tak aby byl **první znak** řetězce **na vrcholu zásobníku** (na zásobník jsou zapisovány od konce - reversal),
2. pravidla, která kontrolují, jestli jsou načítaný znak a znak na zásobníku stejný.

Modely pro syntaktickou analýzu zdola nahoru

RZA $M = (Q, \Sigma, \Gamma, R, s, \#, F)$ ($\#$ je počáteční symbol na zásobníku, s je počáteční stav) a **BKG** $G = (N, T, P, S)$.

1. **M** obsahuje **shiftovací pravidla**, která **přesouvají** vstupní symboly **na zásobník**. (tvorba: Pro každé $t \in \Sigma$: přidej $st \rightarrow ts$ do R .)
2. **M** obsahuje **redukční pravidla**, která simulují **aplikaci gramatických pravidel** pozpátku. (tvorba: Pro každé $A \in T$ přidej $xs \rightarrow As$ do R)
3. **M** také obsahuje **speciální pravidlo** $\#Ss \rightarrow f$, pomocí kterého provede **M** přechod do **koncového stavu**.

Jinak řečeno:

Načítáním symbolů se snažíme dostat do takového stavu, že na zásobníku jsou pouze symboly $\#$ (dno zásobníku) a **S** (kořenový neterminál derivačního stromu, syntaktickou analýzou se snažíme dokázat, že použitím vhodných pravidel se lze z S dostat ke čtenému řetězci). Potřebujeme k tomu dva druhy pravidel:

1. Pravidla, která si **zapamatují již přečtené znaky** (tj. uloží je na zásobník),

2. pravidla, která **identifikují správné posloupnosti zapamatovaných znaků a neterminálů** (tj. symbolů na zásobníku) a přepíše je na neterminál.

odkazy:

- Definice z IFJ: IFJ Teorie

Zdroje

- SZZ okruh 22 — studijní materiály FIT BUT (szz-22.docx). Obrázky: media/szz-22/.

Navigace: [\[\[index | • Přehled okruhů\]\]](#) · [\[\[okruhy/21-regularni-jazyky | 21. Regulární jazyky\]\]](#) · další: [\[\[okruhy/23-struktura-prekladace | 23. Struktura překladače\]\]](#) [□](#)